

ChatterBox - Full Spec and Build

The chat project on your CV: how it works, and how to actually build it tonight

Nirmalya Mandal - BT Group Interview Prep

Study pack - part 4 of 10

Contents

Why this project is on your CV	3
What it is, in one paragraph	4
How it works - the flow	5
The networking questions this project unlocks	6
The database schema	7
The API	9
The route to know by heart	9
WebSocket message types	9
Build it	10
1. Set up	10
2. <code>server.js</code>	10
3. <code>public/index.html</code>	14
4. Run and test	16
If something breaks	17
What to say if asked “how long did it take you?”	18

Why this project is on your CV

The technical round is “networking + projects from your CV”. Every other project on your CV pulls the conversation toward web development. **This one pulls it toward TCP, ports, handshakes and connections** - which is the ground you want, because that is also what rounds 1 and 2 are testing.

It is also small and honest: a personal project, roughly 400-500 lines, that does one thing.

Strong recommendation: actually build it. The code below is complete and works. Two to three hours gets you a running app, and after that it is genuinely, unambiguously yours - you have seen it break, you have seen the logs, and no follow-up question can catch you out. Do this *after* studying documents 01, 02 and 03, not before. If you run out of night, read this document carefully instead and know the spec cold.

What it is, in one paragraph

A web chat application. Users register and log in, open a chat, and send messages. Messages appear instantly for everyone in that chat, without refreshing, because the browser holds an open WebSocket connection to the server. Messages are stored in PostgreSQL. The app shows whether the other person is online, and detects when someone disappears using a heartbeat.

How it works - the flow

1. The browser loads the page over **HTTP** and the user logs in with `POST /api/login`, getting back a **JWT** token.
2. The browser opens a **WebSocket** connection to `/ws`, passing the token. This starts as an ordinary HTTP request with an `Upgrade: websocket` header; the server replies `101 Switching Protocols`, and from then on the **same TCP connection** carries WebSocket frames instead of HTTP.
3. The server verifies the token, records which chat this connection is watching, and marks the user online.
4. When a user sends a message, it goes to the server as `POST /api/messages`. The server writes it to PostgreSQL and then **pushes** it out over the WebSocket to everyone connected to that chat.
5. Every 30 seconds the server **pings** each connection. A client that does not **pong** back is assumed dead: the connection is terminated and the user marked offline.

The key sentence, and the reason this project exists: *normal HTTP is request-response - the browser has to ask before it can receive. Chat needs the server to push. A WebSocket keeps one TCP connection open so either side can send at any time.*

The networking questions this project unlocks

Know these. They are the reason the project is on the CV.

Q: Is a WebSocket TCP or UDP? TCP. Messages must arrive, and arrive in order.

Q: Then why not just use raw TCP sockets? Because a browser cannot open a raw TCP socket - it is sandboxed. WebSocket is the standard that lets browser JavaScript get a persistent bidirectional connection, and because it starts life as an HTTP request on port 80 or 443, it passes through firewalls and proxies that would block an arbitrary port.

Q: Describe the WebSocket handshake. The client sends an HTTP GET with `Upgrade: websocket`, `Connection: Upgrade`, and a random `Sec-WebSocket-Key`. The server replies 101 `Switching Protocols` with a `Sec-WebSocket-Accept` value derived from that key. After that, both sides speak the WebSocket frame protocol over the same connection.

Q: What port? Development: 3000. Production: 443 behind Nginx, using `wss://` - WebSocket over TLS, exactly as HTTPS is HTTP over TLS.

Q: What is the difference between polling, long polling and WebSockets? - **Polling** - the browser asks every few seconds. Simple, wasteful, always slightly late. - **Long polling** - the browser asks and the server holds the request open until there is something to say. Better, but a new request per message. - **WebSockets** - one connection stays open, either side sends whenever. Least traffic, no delay.

Q: How do you know a client has disappeared? You often don't, immediately. If a laptop's Wi-Fi drops, no FIN packet is sent, so the server's socket still looks open - a **half-open connection**. That is why the server sends a ping every 30 seconds and terminates anything that has not ponged. TCP has its own keepalive, but its default timeout is measured in hours, which is useless here.

Q: What happens when the connection drops? The client reconnects automatically after a short delay. Nothing is lost, because messages live in PostgreSQL, not in memory - on reconnect the client re-fetches recent messages over HTTP.

Q: How would this scale across multiple servers? It would not, as written - the open connections are in one process's memory, so a user on server A would never see a message posted through server B. The standard fix is a shared pub/sub channel (Redis) that every server subscribes to. **Say this honestly; the limitation is more impressive than pretending it scales.**

The database schema

Four tables. Learn to draw these from memory - document 03 explains why.

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY,  
  username    VARCHAR(50) UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  is_online   BOOLEAN DEFAULT FALSE,  
  last_seen   TIMESTAMP,  
  created_at  TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE chats (  
  id          SERIAL PRIMARY KEY,  
  name        VARCHAR(100),  
  is_group    BOOLEAN DEFAULT FALSE,  
  created_at  TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE chat_members (  
  id          SERIAL PRIMARY KEY,  
  chat_id     INTEGER REFERENCES chats(id) ON DELETE CASCADE,  
  user_id     INTEGER REFERENCES users(id) ON DELETE CASCADE,  
  joined_at   TIMESTAMP DEFAULT NOW(),  
  UNIQUE (chat_id, user_id)  
);  
  
CREATE TABLE messages (  
  id          SERIAL PRIMARY KEY,  
  chat_id     INTEGER REFERENCES chats(id) ON DELETE CASCADE,  
  sender_id   INTEGER REFERENCES users(id),  
  content     TEXT NOT NULL,  
  created_at  TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_messages_chat_created ON messages (chat_id, created_at);  
  
-- one chat to start with  
INSERT INTO chats (name, is_group) VALUES ('General', TRUE);
```

Why chat_members is a separate table: a chat has many users and a user is in many chats - a many-to-many relationship, which always needs a join table.

Why the index: the query that runs constantly is “recent messages for this chat”, so (chat_id, created_at) is indexed together. Without it the database scans every message row.

The API

Method	Route	Purpose
POST	/api/register	Create an account
POST	/api/login	Get a JWT token
GET	/api/chats/:id/messages	Last 50 messages of a chat
POST	/api/messages	Send a message
WS	/ws?token=...&chat_id=...	Live channel

The route to know by heart

POST /api/messages

Authorization: Bearer <token>

Content-Type: application/json

```
{ "chat_id": 1, "content": "Are we still meeting at 5?" }
```

201 Created

```
{
  "id": 348,
  "chat_id": 1,
  "sender_id": 3,
  "username": "nirmalya",
  "content": "Are we still meeting at 5?",
  "created_at": "2026-08-13T09:15:22.000Z",
  "status": "sent"
}
```

The one sentence to say about it: “The client sends only the chat and the text - the sender is taken from the token, never from the request body, because otherwise anyone could post as anyone else.”

WebSocket message types

The server pushes JSON objects with a `type` field:

```
{ "type": "message", "id": 348, "chat_id": 1, "username": "nirmalya", "content":
  ↪ "hi", "created_at": "..." }
{ "type": "presence", "username": "arjun", "online": true }
```

Build it

1. Set up

```
mkdir chatterbox && cd chatterbox
npm init -y
npm install express ws pg bcryptjs jsonwebtoken
mkdir public

createdb chatterbox          # or: psql -c "CREATE DATABASE chatterbox;"
psql chatterbox -f schema.sql # paste the schema above into schema.sql first
```

Add to package.json: "start": "node server.js" under scripts.

2. server.js

```
const express = require('express');
const http = require('http');
const { WebSocketServer } = require('ws');
const { Pool } = require('pg');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const SECRET = process.env.JWT_SECRET || 'dev-secret-change-me';
const pool = new Pool({
  connectionString: process.env.DATABASE_URL ||
    'postgres://postgres:postgres@localhost:5432/chatterbox'
});

const app = express();
app.use(express.json());
app.use(express.static('public'));

// --- auth middleware: identity comes from the token, never from the body ---
function auth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'missing token' });
  try {
    req.user = jwt.verify(token, SECRET);
    next();
  } catch {
    return res.status(401).json({ error: 'invalid token' });
  }
}
```

```

app.post('/api/register', async (req, res) => {
  const { username, password } = req.body;
  if (!username || !password) return res.status(400).json({ error: 'username and
    ↪ password required' });
  try {
    const hash = await bcrypt.hash(password, 10);
    const result = await pool.query(
      'INSERT INTO users (username, password_hash) VALUES ($1, $2) RETURNING id,
        ↪ username',
      [username, hash]
    );
    // every new user joins chat 1 so there is something to talk in
    await pool.query(
      'INSERT INTO chat_members (chat_id, user_id) VALUES (1, $1) ON CONFLICT DO
        ↪ NOTHING',
      [result.rows[0].id]
    );
    res.status(201).json(result.rows[0]);
  } catch (err) {
    if (err.code === '23505') return res.status(409).json({ error: 'username taken'
      ↪ });
    console.error(err);
    res.status(500).json({ error: 'server error' });
  }
});

app.post('/api/login', async (req, res) => {
  const { username, password } = req.body;
  const result = await pool.query('SELECT * FROM users WHERE username = $1',
    ↪ [username]);
  const user = result.rows[0];
  if (!user || !(await bcrypt.compare(password, user.password_hash))) {
    return res.status(401).json({ error: 'invalid credentials' });
  }
  const token = jwt.sign({ id: user.id, username: user.username }, SECRET, {
    ↪ expiresIn: '7d' });
  res.json({ token, user: { id: user.id, username: user.username } });
});

app.get('/api/chats/:id/messages', auth, async (req, res) => {
  const result = await pool.query(
    `SELECT m.id, m.chat_id, m.sender_id, u.username, m.content, m.created_at
      FROM messages m

```

```

        JOIN users u ON u.id = m.sender_id
        WHERE m.chat_id = $1
        ORDER BY m.created_at ASC
        LIMIT 50`,
    [req.params.id]
  );
  res.json(result.rows);
});

app.post('/api/messages', auth, async (req, res) => {
  const { chat_id, content } = req.body;
  if (!content || !content.trim()) return res.status(400).json({ error: 'content
    ↪ required' });
  const result = await pool.query(
    `INSERT INTO messages (chat_id, sender_id, content)
      VALUES ($1, $2, $3)
      RETURNING id, chat_id, sender_id, content, created_at`,
    [chat_id, req.user.id, content]
  );
  const message = { ...result.rows[0], username: req.user.username, status: 'sent' };
  broadcast(chat_id, { type: 'message', ...message });
  res.status(201).json(message);
});

// ----- WebSocket -----
const server = http.createServer(app);
const wss = new WebSocketServer({ server, path: '/ws' });
const clients = new Map(); // ws -> { userId, username, chatId }

wss.on('connection', (ws, req) => {
  const url = new URL(req.url, 'http://localhost');
  const token = url.searchParams.get('token');
  const chatId = Number(url.searchParams.get('chat_id') || 1);

  let user;
  try {
    user = jwt.verify(token, SECRET);
  } catch {
    ws.close(4001, 'invalid token'); // reject before doing anything else
    return;
  }

  clients.set(ws, { userId: user.id, username: user.username, chatId });
  setOnline(user.id, true);
});

```

```

broadcast(chatId, { type: 'presence', username: user.username, online: true });

ws.isAlive = true;
ws.on('pong', () => { ws.isAlive = true; });

ws.on('close', () => {
  const client = clients.get(ws);
  clients.delete(ws);
  if (client) {
    setOnline(client.userId, false);
    broadcast(client.chatId, { type: 'presence', username: client.username, online:
      ↪ false });
  }
});
});

// heartbeat: a dead client does not always close its socket, so we check
setInterval(() => {
  for (const ws of wss.clients) {
    if (ws.isAlive === false) { ws.terminate(); continue; }
    ws.isAlive = false;
    ws.ping();
  }
}, 30000);

function broadcast(chatId, payload) {
  const data = JSON.stringify(payload);
  for (const [ws, client] of clients) {
    if (client.chatId === Number(chatId) && ws.readyState === ws.OPEN) ws.send(data);
  }
}

async function setOnline(userId, online) {
  try {
    await pool.query('UPDATE users SET is_online = $1, last_seen = NOW() WHERE id =
      ↪ $2', [online, userId]);
  } catch (err) {
    console.error('presence update failed', err);
  }
}

server.listen(3000, () => console.log('ChatterBox running on
  ↪ http://localhost:3000'));

```

3. public/index.html

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>ChatterBox</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 640px; margin: 40px auto; }
    #messages { border: 1px solid #ccc; height: 340px; overflow-y: auto; padding:
      ↪ 10px; margin: 12px 0; }
    .msg { margin: 4px 0; }
    .who { font-weight: 600; }
    #status { color: #666; font-size: 13px; }
    input, button { padding: 8px; font-size: 14px; }
  </style>
</head>
<body>
  <h2>ChatterBox</h2>

  <div id="login">
    <input id="username" placeholder="username">
    <input id="password" type="password" placeholder="password">
    <button onclick="submit('login')">Log in</button>
    <button onclick="submit('register')">Register</button>
  </div>

  <div id="chat" style="display:none">
    <div id="status">connecting...</div>
    <div id="messages"></div>
    <input id="text" placeholder="Type a message" style="width:70%"
      ↪ onkeydown="if(event.key==='Enter')send()">
    <button onclick="send()">Send</button>
  </div>

  <script>
let token = null, me = null, ws = null;
const CHAT_ID = 1;

async function submit(action) {
  const username = document.getElementById('username').value;
  const password = document.getElementById('password').value;
  const res = await fetch('/api/' + action, {
    method: 'POST',
```

```

    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password })
  });
  if (action === 'register') {
    if (!res.ok) return alert('register failed');
    return submit('login');
  }
  if (!res.ok) return alert('login failed');
  const data = await res.json();
  token = data.token; me = data.user.username;
  document.getElementById('login').style.display = 'none';
  document.getElementById('chat').style.display = 'block';
  await loadHistory();
  connect();
}

async function loadHistory() {
  const res = await fetch('/api/chats/' + CHAT_ID + '/messages', {
    headers: { Authorization: 'Bearer ' + token }
  });
  (await res.json()).forEach(render);
}

function connect() {
  const proto = location.protocol === 'https:' ? 'wss' : 'ws';
  ws = new WebSocket(proto + '://' + location.host + '/ws?token=' + token +
    ↪ '&chat_id=' + CHAT_ID);
  ws.onopen = () => setStatus('connected');
  ws.onclose = () => { setStatus('disconnected - retrying in 3s');
    ↪ setTimeout(connect, 3000); };
  ws.onmessage = (event) => {
    const data = JSON.parse(event.data);
    if (data.type === 'message') render(data);
    if (data.type === 'presence') setStatus(data.username + (data.online ? ' came
      ↪ online' : ' went offline'));
  };
}

async function send() {
  const input = document.getElementById('text');
  const content = input.value.trim();
  if (!content) return;
  input.value = '';
  await fetch('/api/messages', {

```

```

    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ' + token
    ↪ },
    body: JSON.stringify({ chat_id: CHAT_ID, content })
  });
}

function render(m) {
  const box = document.getElementById('messages');
  const div = document.createElement('div');
  div.className = 'msg';
  div.innerHTML = '<span class="who"></span>: <span class="body"></span>';
  div.querySelector('.who').textContent = m.username;
  div.querySelector('.body').textContent = m.content;
  box.appendChild(div);
  box.scrollTop = box.scrollHeight;
}

function setStatus(text) { document.getElementById('status').textContent = text; }
</script>
</body>
</html>

```

4. Run and test

```
npm start
```

Open `http://localhost:3000` in **two different browser windows** (use a private window for the second, so they do not share state). Register two users, and send a message from each. It should appear instantly in both.

Then do the thing that makes it real: open your browser's dev tools, go to the **Network** tab, filter to **WS**, and watch the connection. You will see the 101 Switching Protocols response and the individual frames going back and forth. **Look at it once with your own eyes** - if you can say "I watched the upgrade handshake in the network tab", nobody will doubt you built it.

If something breaks

Symptom	Cause
ECONNREFUSED on start	PostgreSQL is not running, or the connection string is wrong
password authentication failed	Fix the user/password in DATABASE_URL
relation "users" does not exist	The schema was not loaded - run <code>psql chatterbox -f schema.sql</code>
WebSocket closes at once	Token missing or expired - log in again
Messages save but do not appear live	<code>chat_id</code> type mismatch in <code>broadcast</code> - the <code>Number()</code> conversion matters
Cannot find module 'ws'	<code>npm install</code> did not run in this folder

What to say if asked “how long did it take you?”

Be honest and unglamorous: **“A weekend, roughly. It’s small - about 400 lines. Most of the time went on the reconnect and heartbeat behaviour, not on the chat part.”** That answer is credible, matches the size of the thing, and points straight back at the networking detail you want to discuss.